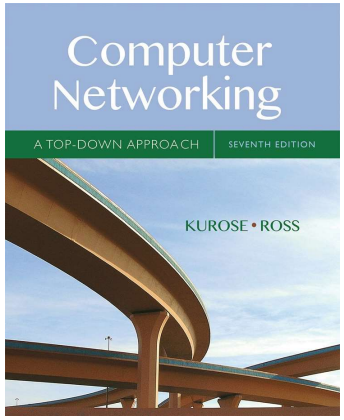
第2章 应用层

中国科学技术大学
自动化系 郑烇
改编自Jim kurose, Keith Ross



Computer Networking: A Top Down Approach
7th edition
Jim Kurose, Keith Ross
Addison-Wesley
April 2016

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 Email
SMTP
POP3
IMAP
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字（Socket）编程
2.9 UDP 套接字编程

第2章：应用层

目标：

- 网络应用的**原理**：网络应用协议的概念和实现方面
  - 传输层的服务模型
  - 客户-服务器模式
  - 对等模式(peer-to-peer)
  - 内容分发网络
- 网络应用的**实例**：互联网流行的应用层协议
  - HTTP
  - FTP
  - SMTP / POP3 / IMAP
  - DNS
- **编程**：网络应用程序
  - Socket API

一些网络应用的例子

- E-mail
- Web
- 文本消息
- 远程登录
- P2P文件共享
- 即时通信
- 多用户网络游戏
- 流媒体(YouTube, Hulu, Netflix)
- Internet 电话
- 实时电视会议
- 社交网络
- 搜索
- ……

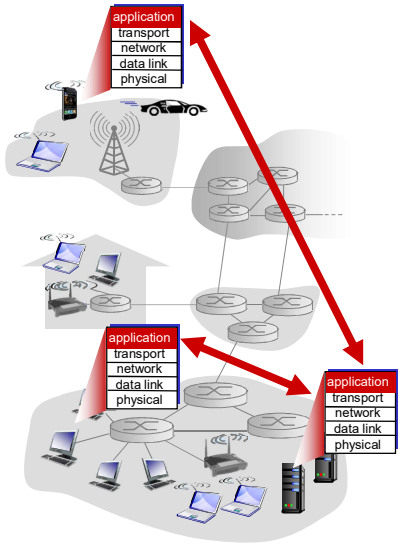
创建一个新的网络应用

编程

- 在不同的端系统上运行
- 通过网络基础设施提供的服务，应用进程彼此通信
- 如Web:
  - Web 服务器软件与浏览器软件通信

网络核心中没有应用层软件

- 网络核心没有应用层功能
- 网络应用只在端系统上存在，快速网络应用开发和部署



第2章：提纲

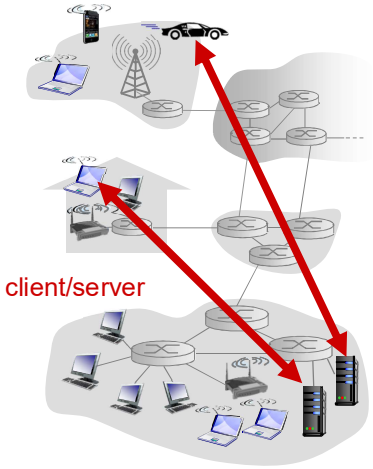
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

网络应用的体系结构

可能的应用架构：

- 客户-服务器模式（C/S:client/server）
- 对等模式(P2P:Peer To Peer)
- 混合体：客户-服务器和对等体系结构

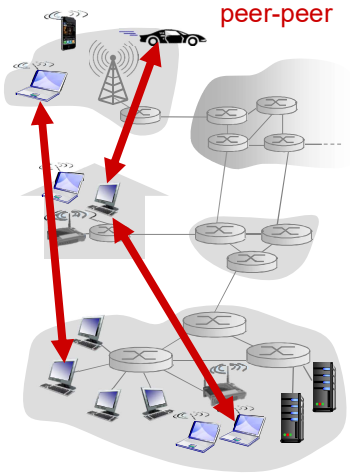
客户-服务器（C/S）体系结构



- 服务器:
  - 一直运行
  - 固定的IP地址和周知的端口号（约定）
  - 扩展性：服务器场
    - 数据中心进行扩展
    - 扩展性差
- 客户端:
  - 主动与服务器通信
  - 与互联网有间歇性的连接
  - 可能是动态IP 地址
  - 不直接与其它客户端通信

对等体（P2P）体系结构

- （几乎）没有一直运行的服务器
- 任意端系统之间可以进行通信
- 每一个节点既是客户端又是服务器
  - 自扩展性-新peer节点带来新的服务能力，当然也带来新的服务请求
- 参与的主机间歇性连接且可以改变IP 地址
  - 难以管理
- 例子: Gnutella，迅雷



C/S和P2P体系结构的混合体

Napster

- 文件搜索：集中
  - 主机在中心服务器上注册其资源
  - 主机向中心服务器查询资源位置
- 文件传输：P2P
  - 任意Peer节点之间

即时通信

- 在线检测：集中
  - 当用户上线时，向中心服务器注册其IP地址
  - 用户与中心服务器联系，以找到其在线好友的位置
- 两个用户之间聊天：P2P

进程通信

进程：在主机上运行的应用程序

- 在同一个主机内，使用进程间通信机制通信（操作系统定义）
- 不同主机，通过交换报文（Message）来通信
  - 使用OS提供的通信服务
  - 按照应用协议交换报文
    - 借助传输层提供的服务

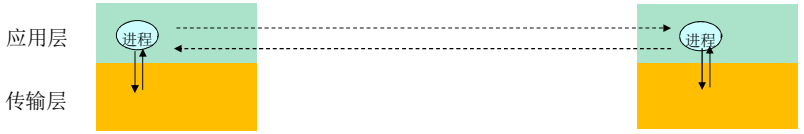
clients, servers

客户端进程：发起通信的进程

服务器进程：等待连接的进程

- 注意：P2P架构的应用也有客户端进程和服务器进程之分

分布式进程通信需要解决的问题

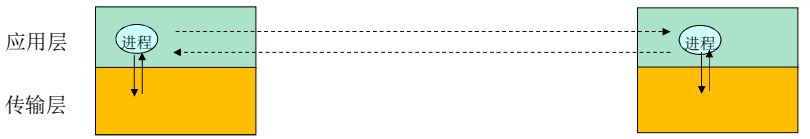


- 问题1：进程标示和寻址问题（服务用户）
- 问题2：传输层-应用层提供服务是如何（服务）
  - 位置：层间界面的SAP （TCP/IP ： socket）
  - 形式：应用程序接口API （TCP/IP ： socket API）
- 问题3：如何使用传输层提供的服务，实现应用进程之间的报文交换，实现应用（用户使用服务）
  - 定义应用层协议：报文格式，解释，时序等
  - 编制程序，使用OS提供的API ，调用网络基础设施提供通信服务传报文，实现应用时序等；

问题1：对进程进行编址（addressing）

- 进程为了接收报文，必须有一个标识
  - 即：SAP（发送也需要标示）
  - 主机：唯一的 32位IP地址
    - 仅仅有IP地址不能够唯一标示一个进程；在一台端系统上有很多应用进程在运行
  - 所采用的传输层协议：TCP or UDP
  - 端口号（Port Numbers）
- 一些知名端口号的例子：
  - HTTP: TCP 80 Mail: TCP25 ftp:TCP 2
- 一个进程：用IP+port标示 端节点
- 本质上，一对主机进程之间的通信由2个端节点构成

问题2：传输层提供的服务-需要穿过层间的信息



- 层间接口必须要携带的信息
  - 要传输的报文（对于本层来说：SDU）
  - 谁传的：自己的应用进程标示：IP+TCP(UDP) 端口
  - 传给谁：对方的应用进程标示：对方的IP+TCP(UDP)端口号
- 传输层实体（tcp或者udp实体）根据这些信息进行TCP报文段（UDP数据报）的封装
  - 源端口号，目标端口号，数据等
  - 将IP地址往下交IP实体，用于封装IP数据报：源IP,目标IP

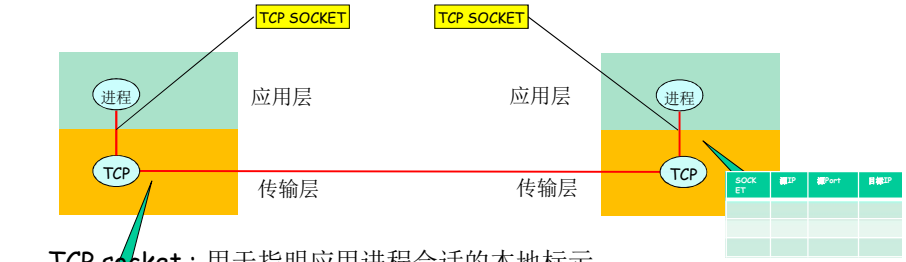
问题2：传输层提供的服务-层间信息的代表

- 如果Socket API 每次传输报文，都携带如此多的信息，太繁琐易错，不便于管理
- 用个代号标示通信的双方或者单方：socket
- 就像OS打开文件返回的句柄一样
  - 对句柄的操作，就是对文件的操作
- TCP socket：
  - TCP服务，两个进程之间的通信需要之前要建立连接
    - 两个进程通信会持续一段时间，通信关系稳定
  - 可以用一个整数表示两个应用实体之间的通信关系，本地标示
  - 穿过层间接口的信息量最小
  - TCP socket：源IP,源端口，目标IP，目标IP,目标端口

TCP之上的套接字（socket）

- 对于使用面向连接服务（TCP）的应用而言，套接字是4元组的一个具有本地意义的标示
  - 4元组：(源IP，源port，目标IP，目标port)
  - 唯一的指定了一个会话（2个进程之间的会话关系）
  - 应用使用这个标示，与远程的应用进程通信
  - 不必在每一个报文的发送都要指定这4元组
  - 就像使用操作系统打开一个文件，OS返回一个文件句柄一样，以后使用这个文件句柄，而不是使用这个文件的目录名、文件名
  - 简单，便于管理

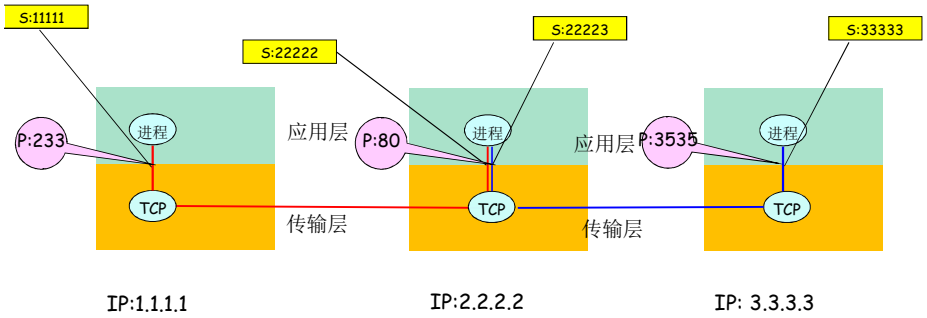
TCP socket



TCP socket：用于指明应用进程会话的本地标示

SOCKET	源IP	源Port	目标IP	目标Port	状态。。

TCP socket



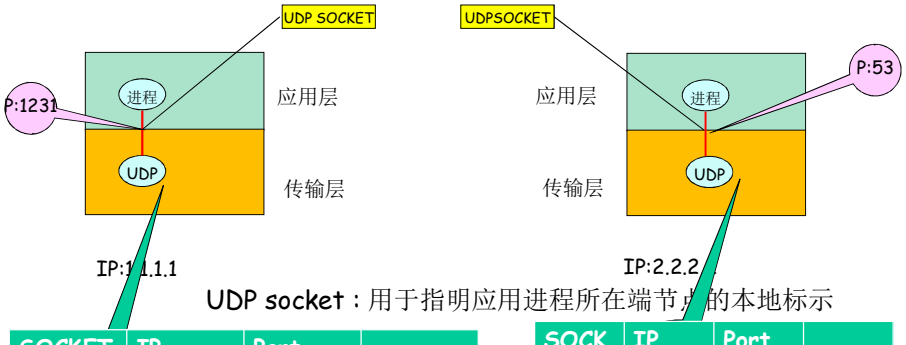
问题2：传输层提供的服务-层间信息代码

- UDP socket：
  - UDP服务，两个进程之间的通信需要之前无需建立连接
    - 每个报文都是独立传输的
    - 前后报文可能给不同的分布式进程
  - 因此，只能用一个整数表示本应用实体的标示
    - 因为这个报文可能传给另外一个分布式进程 ·1
  - 穿过层间接口的信息大小最小
  - UDP socket：本IP,本端口
  - 但是传输 报文时：必须要提供对方IP，port
    - 接收报文时： 传输层需要上传对方的IP，port

UDP之上的套接字（socket）

- 对于使用无连接服务（UDP）的应用而言，套接字是2元组的一个具有本地意义的标示
  - 2元组：IP，port （源端指定）
  - UDP套接字指定了应用所在的一个端节点（end point）
  - 在发送数据报时，采用创建好的本地套接字（标示ID），就不必在发送每个报文中指明自己所采用的ip和port
  - 但是在发送报文时，必须要指定对方的ip和udp port(另外一个段节点)

UDP socket



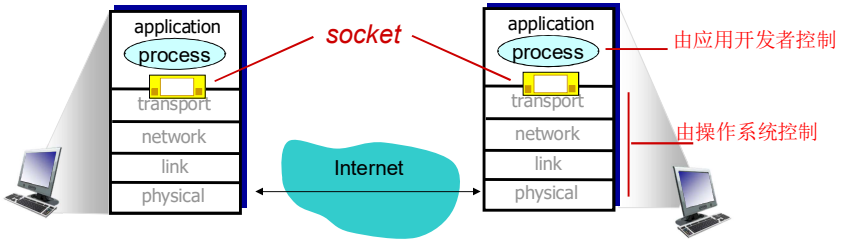
UDP socket：用于指明应用进程所在端节点的本地标示

SOCKET	IP	Port	

SOCKET	IP	Port	

套接字（Socket）

- 进程向套接字发送报文或从套接字接收报文
- 套接字 <-> 门户
  - 发送进程将报文**推出**门户，发送进程依赖于传输层设施在另外一侧的门将报文交付给接受进程
  - 接收进程从另外一端的门户**收到**报文（依赖于传输层设施）



问题3：如何使用传输层提供的服务实现应用



- 定义应用层协议：报文格式，解释，时序等
- 编制程序，通过API调用网络基础设施提供通信服务传报文，解析报文，实现应用时序等

应用层协议

- 定义了：运行在不同端系统上的应用**进程**如何相互交换报文
  - 交换的报文类型：请求和应答报文
  - 各种报文类型的语法：报文中的各个字段及其描述
  - 字段的语义：即字段取值的含义
  - 进程何时、如何发送报文及对报文进行响应的规则
- 应用协议仅仅是应用的一个组成部分
  - Web应用：HTTP协议，web客户端，web服务器，HTML

公开协议：
- 由RFC文档定义
- 允许互操作
- 如HTTP, SMTP

专用（私有）协议：
- 协议不公开
- 如：Skype

应用需要传输层提供什么样的服务？
如何描述传输层的服务？

数据丢失率
- 有些应用则要求100%的可靠数据传输（如文件）
- 有些应用（如音频）能容忍一定比例以下的数据丢失

延迟
- 一些应用 出于有效性考虑，对数据传输有严格的时间限制
  - Internet 电话、交互式游戏
  - 延迟、延迟差

吞吐
- 一些应用（如多媒体）必须需要最小限度的吞吐，从而使得应用能够有效运转
- 一些应用能充分利用可供使用的吞吐(弹性应用)

安全性
- 机密性
- 完整性
- 可认证性（鉴别）

常见应用对传输服务的要求

应用	数据丢失率	吞吐	时间敏感性
文件传输	不能丢失	弹性	不
e-mail	不能丢失	弹性	不
Web 文档	不能丢失	弹性	不
实时音视频	容忍丢失	音频： 5kbps-1Mbps 视频：0kbps-5Mbps	是，100ms
存储音视频	容忍丢失	同上	是，几秒
交互式游戏	容忍丢失	几kbps ~10kbps	是，100ms
即时讯息	不能丢失	弹性	是和不是

Internet 传输层提供的服务

TCP 服务：
- 可靠的传输服务
- 流量控制：发送方不会淹没接受方
- 拥塞控制：当网络出现拥塞时，能抑制发送方
- 不能提供的服务：时间保证、最小吞吐保证和安全
- 面向连接：要求在客户端进程和服务器进程之间建立连接

UDP 服务：
- 不可靠数据传输
- 不提供的服务：可靠，流量控制、拥塞控制、时间、带宽保证、建立连接

Q: 为什么要有 UDP?

UDP存在的必要性

- 能够区分不同的进程，而IP服务不能
  - 在IP提供的主机到主机端到端功能的基础上，区分了主机的应用进程
- 无需建立连接，省去了建立连接时间，适合事务性的应用
- 不做可靠性的工作，例如检错重发，适合那些对实时性要求比较高而对正确性要求不高的应用
  - 因为为了实现可靠性（准确性、保序等），必须付出时间代价（检错重发）
- 没有拥塞控制和流量控制，应用能够按照设定的速度发送数据
  - 而在TCP上面的应用，应用发送数据的速度和主机向网络发送的实际速度是不一致的，因为有流量控制和拥塞控制

Internet应用及其应用层协议和传输协议

应用	应用层协议	下层的传输协议
e-mail	SMTP [RFC 2821]	TCP
远程终端访问	Telnet [RFC 854]	TCP
Web	HTTP [RFC 2616]	TCP
文件传输	FTP [RFC 959]	TCP
流媒体	专用协议 (如RealNetworks)	TCP 或 UDP
Internet电话	专用协议 (如Net2Phone)	TCP或UDP

安全TCP

TCP & UDP

- 都没有加密
- 明文通过互联网传输，甚至密码

SSL

- 在TCP上面实现，提供加密的TCP连接
- 私密性
- 数据完整性
- 端到端的鉴别

SSL在应用层

- 应用采用SSL库，SSL库使用TCP通信

SSL socket API

- 应用通过API将明文交给socket，SSL将其加密在互联网上传输
- 详见第8章

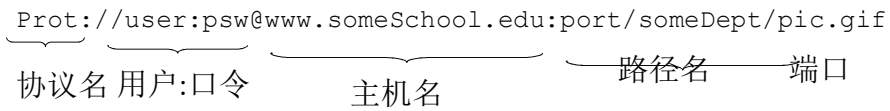
第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

Web 与 HTTP

一些术语

- Web页：由一些对象组成
- 对象可以是HTML文件、JPEG图像、Java小程序、声音剪辑文件等
- Web页含有一个基本的HTML文件，该基本HTML文件又包含若干对象的引用（链接）
- 通过URL对每个对象进行引用
  - 访问协议，用户名，口令字，端口，目录文件等；
- URL格式:

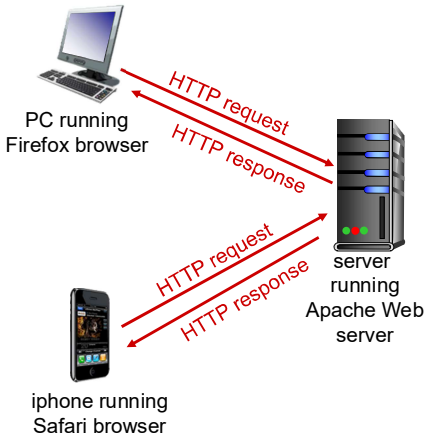


HTTP概况

HTTP: 超文本传输协议

- Web的应用层协议
- 客户/服务器模式
  - 客户: 请求、接收和显示Web对象的浏览器
  - 服务器: 对请求进行响应，发送对象的Web服务器

- HTTP 1.0: RFC 1945
- HTTP 1.1: RFC 2068



HTTP概况(续)

使用TCP:

- 客户发起一个与服务器的TCP连接 (建立套接字)，端口号为 80
- 服务器接受客户的TCP连接
- 在浏览器(HTTP客户端)与 Web服务器(HTTP服务器 server)交换HTTP报文 (应用层协议报文)
- TCP连接关闭

HTTP是无状态的

- 服务器并不维护关于客户的任何信息

维护状态的协议很复杂！

- 必须维护历史信息(状态)
- 如果服务器/客户端死机，它们的状态信息可能不一致，二者的信息必须是一致
- 无状态的服务器能够支持更多的客户端

HTTP连接

非持久HTTP

- 最多只有一个对象在TCP连接上发送
- 下载多个对象需要多个TCP连接
- HTTP/1.0使用非持久连接

持久HTTP

- 多个对象可以在一个（在客户端和服务器之间的）TCP连接上传输
- HTTP/1.1 默认使用持久连接

非持久HTTP连接

假设用户输入URL

```
www.someSchool.edu/someDept/home.index
```

(包含文本和10个就jpeg图像的引用)

1a. HTTP客户端在端口号80发起一个到服务器www.someSchool.edu的连接

1b. 位于主机www.someSchool.edu的HTTP服务器在80号端口等待连接，接受连接并通知客户端

2. HTTP客户端向TCP连接的套接字发送HTTP请求报文，报文表示客户端需要对象someDepartment/home.index

3. HTTP服务器接收到请求报文，检索出被请求的对象，将对象封装在一个响应报文，并通过其套接字象客户端发送

时间

非持久HTTP (续)

4. HTTP关闭TCP连接。

5. HTTP客户端收到包含html文件的响应报文，并显示html。然后对html文件进行检查，找到10引用对象

时间

6. 对10jpeg对象，重复1-5步

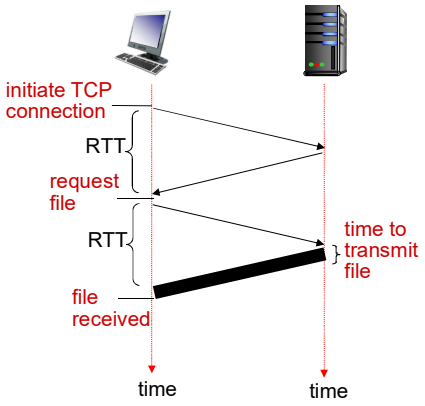
响应时间模型

往返时间RTT（round-trip time）：一个小的分组从客户端到服务器，在回到客户端的时间（传输时间忽略）

响应时间：

- 一个RTT用来发起TCP连接
- 一个 RTT用来HTTP请求并等待HTTP响应
- 文件传输时间

共：2RTT+传输时间



持久HTTP

非持久HTTP的缺点：

- 每个对象要2个 RTT
- 操作系统必须为每个TCP连接分配资源
- 但浏览器通常打开并行TCP连接，以获取引用对象

持久HTTP

- 服务器在发送响应后，仍保持TCP连接
- 在相同客户端和服务器之间的后续请求和响应报文通过相同的连接进行传送
- 客户端在遇到一个引用对象的时候，就可以尽快发送该对象的请求

非流水方式的持久HTTP：

- 客户端只能在收到前一个响应后才能发出新的请求
- 每个引用对象花费一个RTT

流水方式的持久HTTP：

- HTTP/1.1的默认模式
- 客户端遇到一个引用对象就立即产生一个请求
- 所有引用（小）对象只花费一个RTT是可能的

HTTP请求报文

- 两种类型的HTTP报文：请求、响应
- HTTP请求报文:
  - ASCII (人能阅读)

请求行 (GET, POST, HEAD命令)

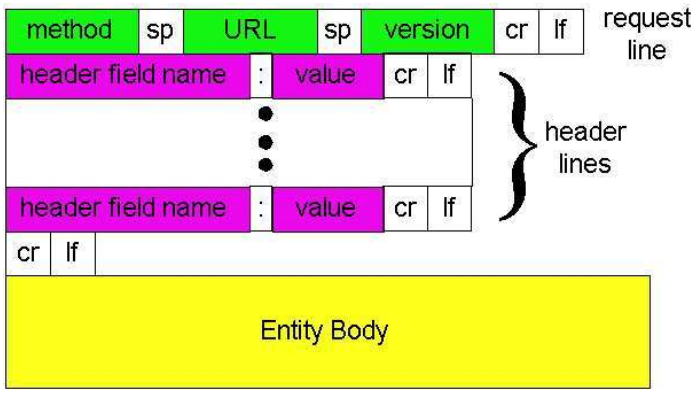
首部行

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language:fr
```

换行回车符，表示报文结束

(一个额外的换行回车符)

HTTP请求报文：通用格式



提交表单输入

Post方式：

- 网页通常包括表单输入
- 包含在实体主体(entity body)中的输入被提交到服务器

URL方式：

- 方法：GET
- 输入通过请求行的URL字段上载

www.somesite.com/animalsearch?monkeys&banana

http://www.baidu.com/s?wd=xx+yy+zzz&cl=3

参数：wd，cl

参数值：XX+YY+zzz，3

方法类型

HTTP/1.0

- GET
- POST
- HEAD
  - 要求服务器在响应报文中不包含请求对象 → 故障跟踪

HTTP/1.1

- GET, POST, HEAD
- PUT
  - 将实体主体中的文件上载到URL字段规定的路径
- DELETE
  - 删除URL字段规定的文件

HTTP响应报文

状态行 (协议版本、状态码和相应状态信息)

首部行

数据，如请求的HTML文件

```
HTTP/1.1 200 OK\r\n
Connection close\r\n
Date: Thu, 06 Aug 1998 12:00:15 GMT\r\r
Server: Apache/1.3.0 (Unix) \r\n
Last-Modified: Mon, 22 Jun 1998 …... \r
Content-Length: 6821\r\n
Content-Type: text/html\r\n
\r\n
\r\n
data data data data data ...
```

HTTP响应状态码

位于服务器→客户端的响应报文中的首行

一些状态码的例子：

200 OK
- 请求成功，请求对象包含在响应报文的后续部分

301 Moved Permanently
- 请求的对象已经被永久转移了：新的URL在响应报文的Location:首部行中指定
- 客户端软件自动用新的URL去获取对象

400 Bad Request
- 一个通用的差错代码，表示该请求不能被服务器解读

404 Not Found
- 请求的文档在该服务上没有找到

505 HTTP Version Not Supported

Trying out HTTP (client side) for yourself

1. Telnet to your favorite Web server:

```
telnet cis.poly.edu 80
```

opens TCP connection to port 80 (default HTTP server port) at cis.poly.edu. anything typed in sent to port 80 at cis.poly.edu

2. type in a GET HTTP request:

```
GET /~ross/ HTTP/1.1
Host: cis.poly.edu
```

by typing this in (hit carriage return twice), you send this minimal (but complete) GET request to HTTP server

3. look at response message sent by HTTP server!

(or use Wireshark to look at captured HTTP request/response)

用户-服务器状态：cookies

大多数主要的门户网站使用 cookies

4个组成部分：

1) 在HTTP响应报文中有一个cookie的首部行

2)在HTTP请求报文含有一个cookie的首部行

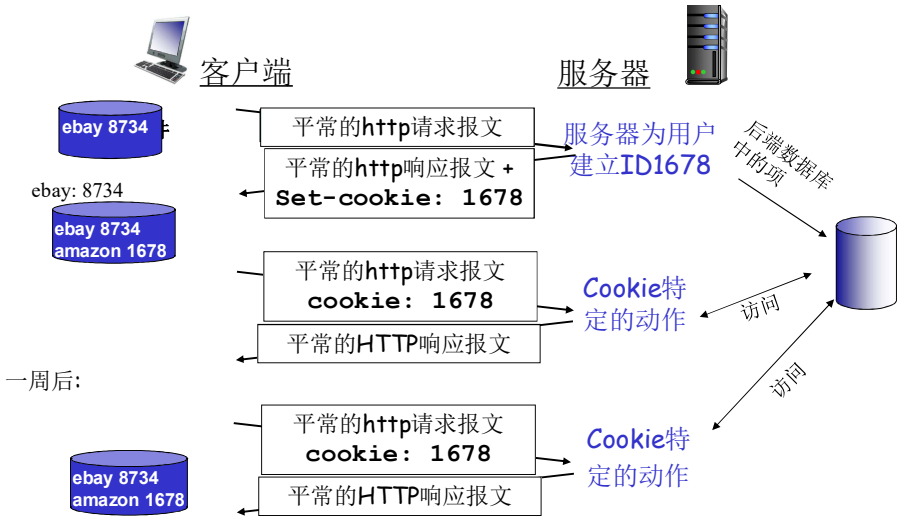
3) 在用户端系统中保留有一个cookie文件，由用户的浏览器管理

4) 在Web站点有一个后端数据库

例子：

- Susan总是用同一个PC使用Internet Explore上网
- 她第一次访问了一个使用了Cookie的电子商务网站
- 当最初的HTTP请求到达服务器时，该Web站点产生一个唯一的ID，并以此作为索引在它的后端数据库中产生一个项

Cookies: 维护状态



Cookies (续)

Cookies能带来什么：

- 用户验证
- 购物车
- 推荐
- 用户状态 (Web e-mail)

如何维持状态：

- 协议端节点：在多个事务上，发送端和接收端维持状态
- cookies：http报文携带状态信息

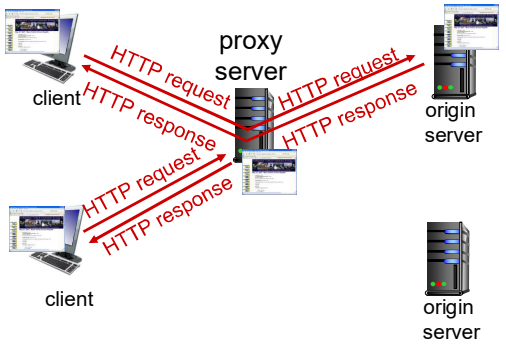
Cookies与隐私：

- Cookies允许站点知道许多关于用户的信息
- 可能将它知道的东西卖给第三方
- 使用重定向和cookie的搜索引擎还能知道用户更多的信息
  - 如通过某个用户在大量站点上的行为，了解其个人浏览方式的大致模式
- 广告公司从站点获得信息

Web缓存 (代理服务器)

目标：不访问**原始**服务器，就满足客户的请求

- 用户设置浏览器： 通过缓存访问Web
- 浏览器将所有的HTTP请求发给缓存
  - 在缓存中的对象：缓存直接返回对象
  - 如对象不在缓存，缓存请求原始服务器，然后再将对象返回给客户端



Web缓存

- 缓存既是客户端又是服务器
- 通常缓存是由ISP安装 (大学、公司、居民区ISP)

为什么要使用Web缓存？

- 降低客户端的请求响应时间
- 可以大大减少一个机构内部网络与Internent接入链路上的流量
- 互联网大量采用了缓存：可以使较弱的ICP也能够有效提供内容

缓存示例

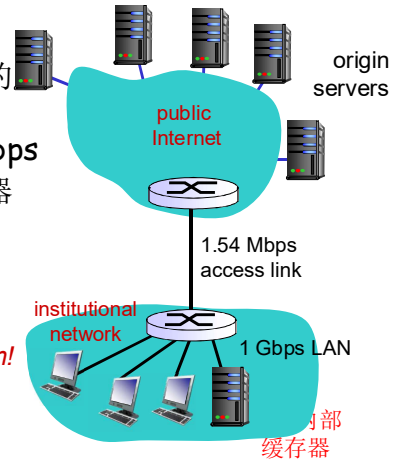
假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps

结果

- LAN的流量强度 = 15%
- 接入链路上的流量强度 = 99%
- 总延时 = LAN延时 + 接入延时 + Internet 延时

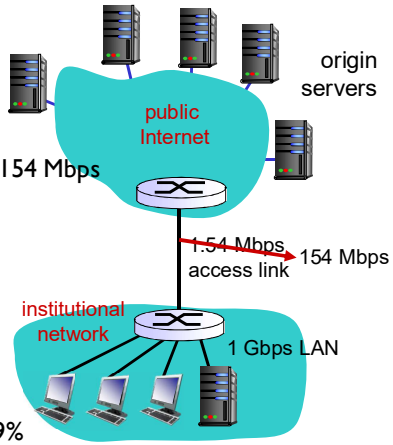
= ms + 分 + 2s



缓存示例：更快的接入链路

假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps



结果

- LAN的流量强度 = 15%
- 接入链路上的流量强度 = 99%
- 总延时 = LAN延时 + 接入延时 + Internet 延时 = ms + 分 + 2s = msecs

9.9%

代价： 增加了接入链路带宽（非常昂贵！）

缓存例子：安装本地缓存

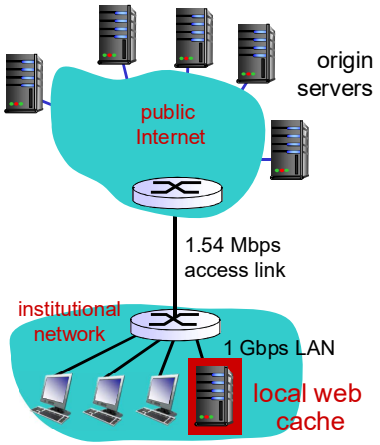
假设

- 平均对象大小 = 100kb
- 机构内浏览器对原始服务器的平均请求率为 = 15请求/s
- 平均到浏览器的速率：1.5Mbps
- 机构内部路由器到原始服务器再返回到路由器的的延时 （Internet 延时）= 2s
- 接入链路带宽：1.54Mbps

结果

- LAN 利用率：15%
- 接入网络利用率： ?
- 总体延迟= ?

How to compute link utilization, delay?

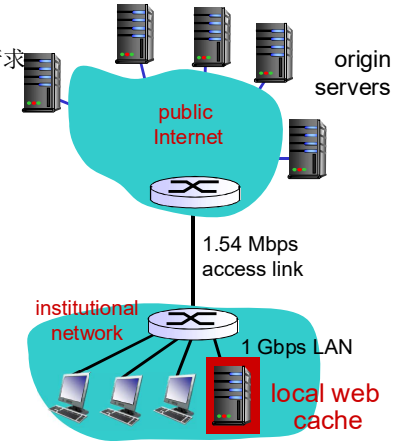


代价： web缓存(廉价!)

缓存例子：安装本地缓存

计算链路利用率，有缓存的延迟：

- 假设缓存命中率0.4
  - 40%请求在缓存中被满足，其他60%的请求需要被原始服务器满足



- 接入链路利用率:
  - 60%的请求采用接入链路
- 进过接入链路到达浏览器的数据速率 = 0.6*1.50 Mbps = .9 Mbps
  - 利用率= 0.9/1.54 = .58
- 总体延迟：
  - = 0.6 *（从原始服务器获取对象的延迟）+0.4 *（从缓存获取对象的延迟）
  - = 0.6 (2.01) + 0.4 (~msecs)
  - = ~ 1.2 secs
  - 比安装154Mbps链路还来得小（而且比较便宜!）

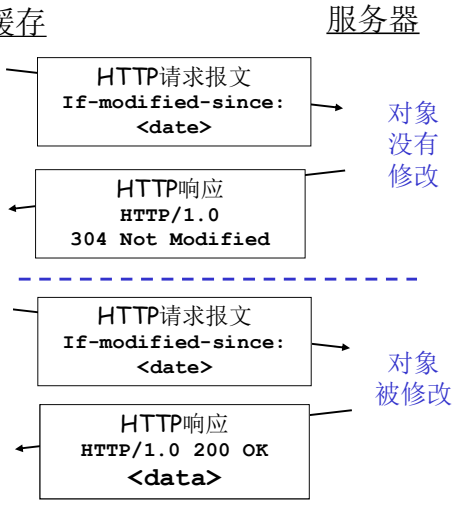
条件GET方法

- 目标：如果缓存器中的对象拷贝是最新的，就不要发送对象
- 缓存器: 在HTTP请求中指定缓存拷贝的日期

```
If-modified-since:
  <date>
```

- 服务器: 如果缓存拷贝陈旧，则响应报文没包含对象:

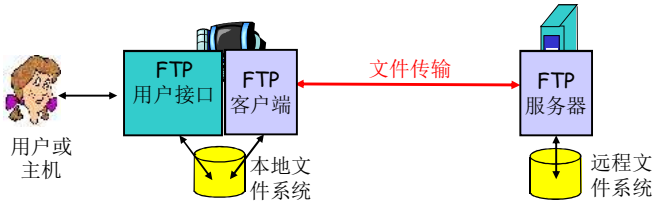
```
HTTP/1.0 304 Not
  Modified
```



第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

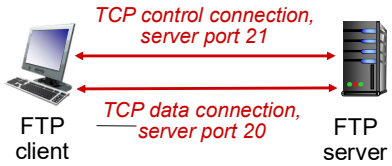
FTP: 文件传输协议



- 向远程主机上传输文件或从远程主机接收文件
- 客户/服务器模式
  - 客户端：发起传输的一方
  - 服务器：远程主机
- ftp: RFC 959
- ftp服务器：端口号为21

FTP: 控制连接与数据连接分开

- FTP客户端与FTP服务器通过端口21联系，并使用TCP为传输协议
- 客户端通过控制连接获得身份确认
- 客户端通过控制连接发送命令浏览远程目录
- 收到一个文件传输命令时，服务器打开一个到客户端的数据连接
- 一个文件传输完成后，服务器关闭连接



- 服务器打开第二个TCP数据连接用来传输另一个文件
- 控制连接： 带外（ “out of band” ）传送
- FTP服务器维护用户的状态信息：当前路径、用户帐户与控制连接对应

有状态

FTP命令、响应

命令样例：

- 在**控制连接**上以*ASCII*文本方式传送
- **`USER username`**
- **`PASS password`**
- **`LIST`**：请服务器返回远程主机当前目录的文件列表
- **`RETR filename`**：从远程主机的当前目录检索文件(gets)
- **`STOR filename`**：向远程主机的当前目录存放文件(puts)

返回码样例：

- 状态码和状态信息 (同HTTP)
- **`331 Username OK, password required`**
- **`125 data connection already open; transfer starting`**
- **`425 Can’ t open data connection`**
- **`452 Error writing file`**

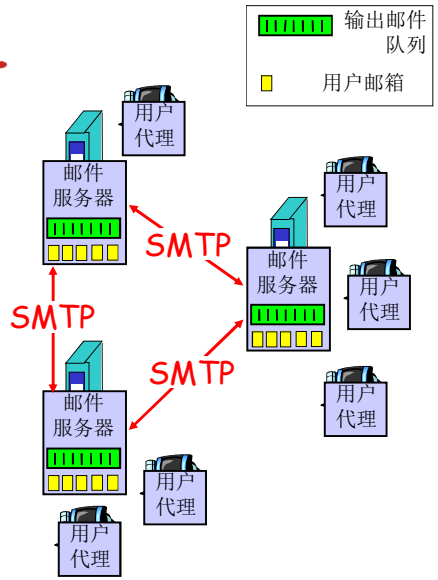
第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

电子邮件（EMail）

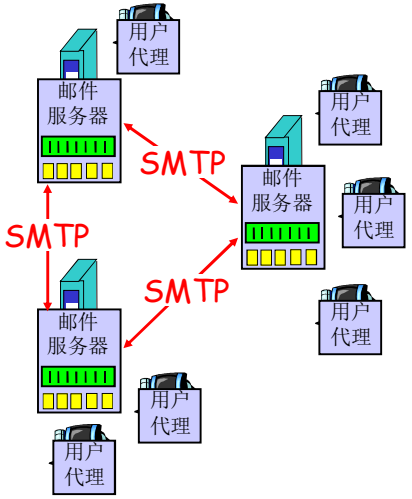
3个主要组成部分：
- 用户代理
- 邮件服务器
- 简单邮件传输协议：SMTP

用户代理
- 又名“邮件阅读器”
- 撰写、编辑和阅读邮件
- 如Outlook、Foxmail
- 输出和输入邮件保存在服务器上



EMail: 邮件服务器

邮件服务器
- 邮箱中管理和维护发送给用户的邮件
- 输出报文队列保持待发送邮件报文
- 邮件服务器之间的SMTP协议：发送email报文
  - 客户：发送方邮件服务器
  - 服务器：接收端邮件服务器

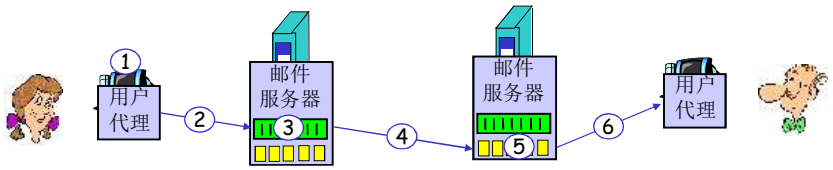


EMail: SMTP [RFC 2821]

- 使用TCP在客户端和服务器之间传送报文，端口号为25
- 直接传输：从发送方服务器到接收方服务器
- 传输的3个阶段
  - 握手
  - 传输报文
  - 关闭
- 命令/响应交互
  - 命令：ASCII文本
  - 响应：状态码和状态信息
- 报文必须为7位ASCII码

举例：Alice给Bob发送报文

1) Alice使用用户代理撰写邮件并发送给 bob@someschool.edu
2) Alice的用户代理将邮件发送到她的邮件服务器；邮件放在报文队列中
3) SMTP的客户端打开到Bob邮件服务器的TCP连接
4) SMTP客户端通过TCP连接发送Alice的邮件
5) Bob的邮件服务器将邮件放到Bob的邮箱
6) Bob调用他的用户代理阅读邮件



简单的SMTP交互

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250  Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```


Try SMTP interaction for yourself:

- `telnet servername 25`
- see 220 reply from server
- enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

SMTP：总结

- SMTP使用持久连接
- SMTP要求报文（首部和主体）为7位ASCII编码
- SMTP服务器使用CRLF.CRLF决定报文的尾部

HTTP比较：

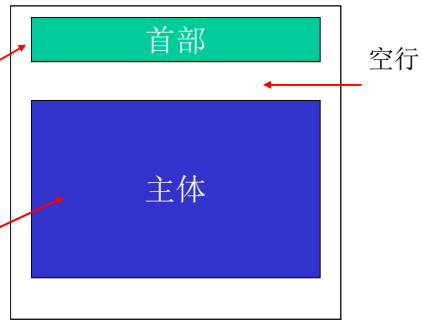
- HTTP：拉（pull）
- SMTP：推（push）
- 二者都是ASCII形式的命令/响应交互、状态码
- HTTP：每个对象封装在各自的响应报文中
- SMTP：多个对象包含在一个报文中

邮件报文格式

SMTP：交换email报文的协议
RFC 822: 文本报文的标准：

- 首部行：如，
  - To:
  - From:
  - Subject:

  与SMTP命令不同！
- 主体
  - 报文，只能是ASCII码字符



报文格式：多媒体扩展

- MIME：多媒体邮件扩展（multimedia mail extension），RFC 2045, 2056
- 在报文首部用额外的行申明MIME内容类型

MIME版本

数据的编码方式

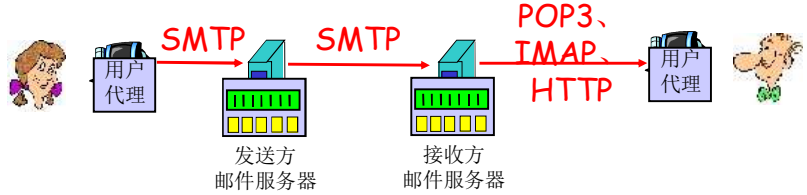
多媒体数据类型、子类型和参数申明

编码好的数据

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.........................
......base64 encoded data
```


邮件访问协议



- SMTP: 传送到接收方的邮件服务器
- 邮件访问协议：从服务器访问邮件
  - POP：邮局访问协议（Post Office Protocol）[RFC 1939]
    - 用户身份确认 (代理<-->服务器) 并下载
  - IMAP：Internet邮件访问协议（Internet Mail Access Protocol）[RFC 1730]
    - 更多特性 (更复杂)
    - 在服务器上处理存储的报文
  - HTTP：Hotmail , Yahoo! Mail等
    - 方便

POP3协议

用户确认阶段

- 客户端命令：
  - **user:** 申明用户名
  - **pass:** 口令
- 服务器响应
  - **+OK**
  - **-ERR**

事物处理阶段，客户端：

- **list:** 报文号列表
- **retr:** 根据报文号检索报文
- **dele:** 删除
- **quit**

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (续) 与 IMAP

POP3（续）

- 先前的例子使用 “下载并删除”模式。
  - 如果改变客户机，Bob不能阅读邮件
- “下载并保留”：不同客户机上为报文的拷贝
- POP3在会话中是无状态的

本地管理文件夹

IMAP

- IMAP服务器将每个报文与一个文件夹联系起来
- 允许用户用目录来组织报文
- 允许用户读取报文组件
- IMAP在会话过程中保留用户状态：
  - 目录名、报文ID与目录名之间映射

远程管理文件夹

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

2.5 DNS(Domain Name System)

- DNS的必要性
  - IP地址标识主机、路由器
  - 但IP地址不好记忆，不便人类使用(没有意义)
  - 人类一般倾向于使用一些有意义的字符串来标识Internet上的设备

    例如：gzheng@ustc.edu.cn 所在的邮件服务器

    www.ustc.edu.cn 所在的web服务器
  - 存在着“字符串”—IP地址的转换的必要性
  - 人类用户提供要访问机器的“字符串”名称
  - 由DNS负责转换成为二进制的网络地址

DNS系统需要解决的问题

- 问题1：如何命名设备
  - 用有意义的字符串：好记，便于人类用使用
  - 解决一个平面命名的重名问题：层次化命名
- 问题2：如何完成名字到IP地址的转换
  - 分布式的数据库维护和响应名字查询
- 问题3：如何维护：增加或者删除一个域，需要在域名系统中做哪些工作

DNS(Domain Name System)的历史

- ARPANET的名字解析解决方案
  - 主机名：没有层次的一个字符串（一个平面）
  - 存在着一个（集中）维护站：维护着一张 主机名-IP地址的映射文件：Hosts.txt
  - 每台主机定时从维护站取文件
- ARPANET解决方案的问题
  - 当网络中主机数量很大时
    - 没有层次的主机名称很难分配
    - 文件的管理、发布、查找都很麻烦

DNS(Domain Name System)总体思路和目标

- DNS的主要思路
  - 分层的、基于域的命名机制
  - 若干分布式的数据库完成名字到IP地址的转换
  - 运行在UDP之上端口号为53的应用服务
  - 核心的Internet功能，但以应用层协议实现
    - 在网络边缘处理复杂性
- DNS主要目的：
  - 实现主机名-IP地址的转换(name/IP translate)
  - 其它目的
    - 主机别名到规范名字的转换：Host aliasing
    - 邮件服务器别名到邮件服务器的正规名字的转换：Mail server aliasing
    - 负载均衡：Load Distribution

问题1：DNS名字空间(The DNS Name Space)

- DNS域名结构
  - 一个层面命名设备会有很多重名
  - NDS采用层次树状结构的 命名方法
  - Internet 根被划为几百个顶级域(top lever domains)
    - 通用的(generic)

      .com; .edu ; .gov ; .int ; .mil ; .net ; .org

      .firm ; .hsop ; .web ; .arts ; .rec ;
    - 国家的(countries)

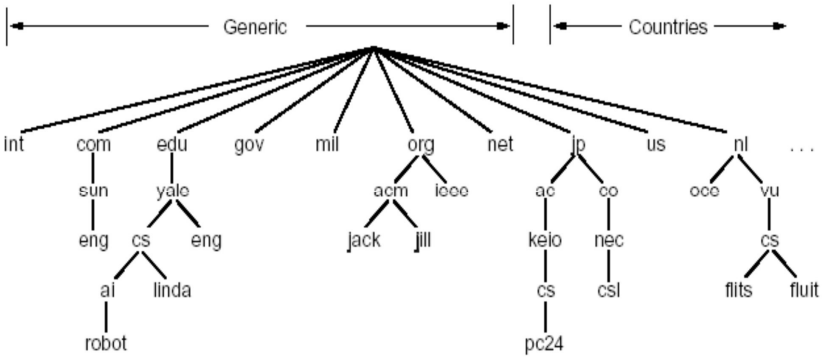
      .cn ; .us ; .nl ; .jp
  - 每个(子)域下面可划分为若干子域(subdomains)
  - 树叶是主机

DNS: 根名字服务器



共有13个根名字服务器

DNS名字空间(The DNS Name Space)



DNS名字空间(The DNS Name Space)

- 域名(Domain Name)
  - 从本域往上，直到树根
  - 中间使用“.”间隔不同的级别
  - 例如：ustc.edu.cn
  - auto.ustc.edu.cn
  - www.auto. ustc.edu.cn
  - 域的域名：可以用于表示一个域
  - 主机的域名：一个域上的一个主机

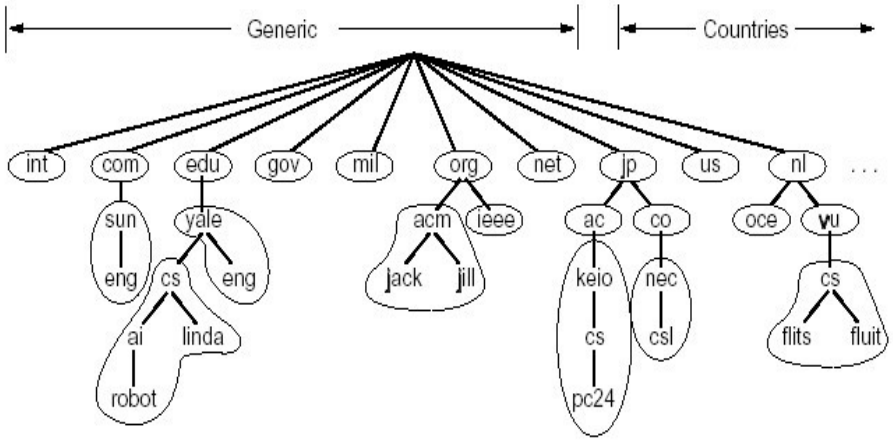
DNS名字空间(The DNS Name Space)

- 域名的管理
  - 一个域管理其下的子域
    .jp 被划分为 ac.jp co.jp
    .cn 被划分为 edu.cn com.cn
  - 创建一个新的域，必须征得它所属域的同意
- 域与物理网络无关
  - 域遵从组织界限，而不是物理网络
    - 一个域的主机可以不在一个网络
    - 一个网络的主机不一定在一个域
  - 域的划分是逻辑的，而不是物理的

问题2：解析问题-名字服务器(Name Server)

- 一个名字服务器的问题
  - 可靠性问题：单点故障
  - 扩展性问题：通信容量
  - 维护问题：远距离的集中式数据库
- 区域(zone)
  - 区域的划分有区域管理者自己决定
  - 将DNS名字空间划分为互不相交的区域，每个区域都是树的一部分
  - 名字服务器：
    - 每个区域都有一个名字服务器：维护着它所管辖区域的权威信息(authoritative record)
    - 名字服务器允许被放置在区域之外，以保障可靠性

名字空间划分为若干区域：Zone



权威DNS服务器：组织机构的DNS服务器， 提供组织机构服务器（如Web和mail）可访问的主机和IP之间的映射
组织机构可以选择实现自己维护或由某个服务提供商来维护

TLD服务器

- 顶级域(TLD)服务器：负责顶级域名（如com, org, net, edu和gov）和所有国家级的顶级域名（如cn, uk, fr, ca, jp ）
  - Network solutions 公司维护com TLD服务器
  - Educause公司维护edu TLD服务器

区域名字服务器维护资源记录

- 资源记录(resource records)
  - 作用：维护 域名-IP地址(其它)的映射关系
  - 位置：Name Server的分布式数据库中
- RR格式：(domain_name, ttl, type,class,Value)
  - Domain_name: 域名
  - Ttl: time to live：生存时间(权威，缓冲记录)
  - Class 类别 ：对于Internet，值为IN
  - Value 值：可以是数字，域名或ASCII串
  - Type 类别：资源记录的类型—见下页

DNS记录

DNS ：保存资源记录(RR)的分布式数据库

RR 格式：(name, value, type, ttl)

- Type=A
  - Name为主机
  - Value为IP地址
- Type=NS
  - Name域名(如foo.com)
  - Value为该域名的权威服务器的域名
- Type=CNAME
  - Name为规范名字的别名
    www.ibm.com 的规范名字为
    servereast.backup2.ibm.com
  - value 为规范名字
- Type=MX
  - Value为name对应的邮件服务器的名字

TTL：生存时间，决定了资源记录应当从缓存中删除的时间

资源记录(resource records)

- 一个例子

```
; Authoritative data for cs.vu.nl
cs.vu.nl.       86400   IN SOA    star boss (952771,7200,7200,2419200,86400)
cs.vu.nl.       86400   IN TXT    "Faculteit Wiskunde en Informatica."
cs.vu.nl.       86400   IN TXT    "Vrije Universiteit Amsterdam."
cs.vu.nl.       86400   IN MX     1 zephyr.cs.vu.nl.
cs.vu.nl.       86400   IN MX     2 top.cs.vu.nl.

flits.cs.vu.nl. 86400   IN HINFO  Sun Unix
flits.cs.vu.nl. 86400   IN A      130.37.16.112
flits.cs.vu.nl. 86400   IN A      192.31.231.165
flits.cs.vu.nl. 86400   IN MX     1 flits.cs.vu.nl.
flits.cs.vu.nl. 86400   IN MX     2 zephyr.cs.vu.nl.
flits.cs.vu.nl. 86400   IN MX     3 top.cs.vu.nl.
www.cs.vu.nl.   86400   IN CNAME  star.cs.vu.nl
ftp.cs.vu.nl.   86400   IN CNAME  zephyr.cs.vu.nl

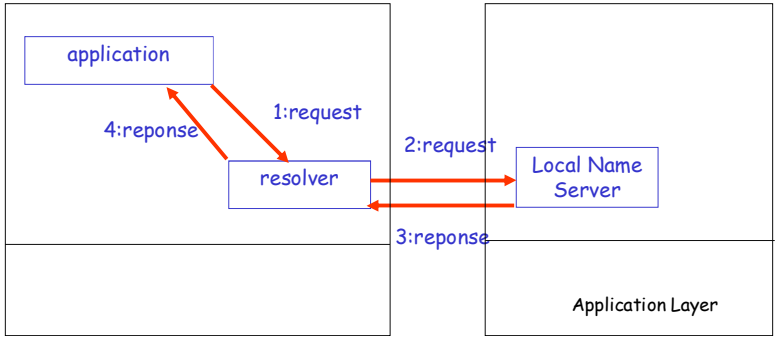
rowboat                 IN A      130.37.56.201
                        IN MX     1 rowboat
                        IN MX     2 zephyr
                        IN HINFO  Sun Unix

little-sister           IN A      130.37.62.23
                        IN HINFO  Mac MacOS

laserjet                IN A      192.31.231.216
                        IN HINFO  "HP Laserjet IIISi" Proprietary
```

DNS(Domain Name System)

- DNS大致工作过程
  - 应用调用 解析器(resolver)
  - 解析器作为客户 向Name Server发出查询报文（封装在UDP段中）
  - Name Server返回响应报文(name/ip)



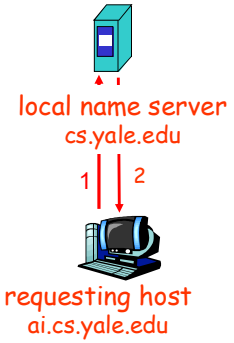
本地名字服务器（Local Name Server）

- 并不严格属于层次结构
- 每个ISP (居民区的ISP、公司、大学）都有一个本地DNS服务器
  - 也称为“默认名字服务器”
- 当一个主机发起一个DNS查询时，查询被送到其本地DNS服务器
  - 起着代理的作用，将查询转发到层次结构中

名字服务器(Name Server)

- 名字解析过程
  - 目标名字在Local Name Server中
    - 情况1：查询的名字在该区域内部
    - 情况2：缓存(cashing)

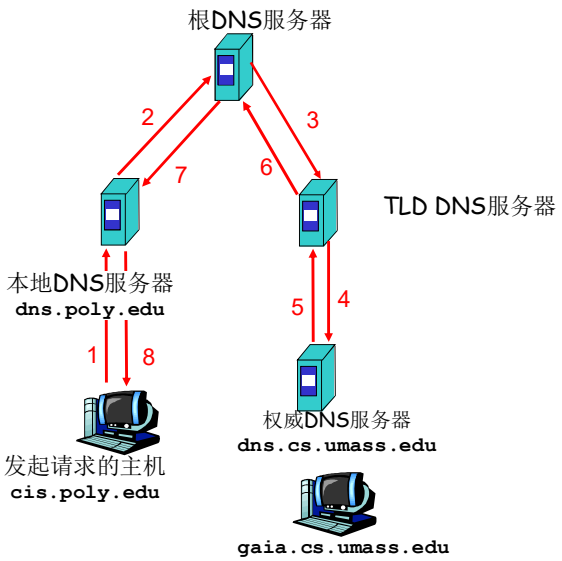
当与本地名字服务器不能解析名字时，联系根名字服务器 顺着根-TLD 一直找到 权威名字服务器



递归查询

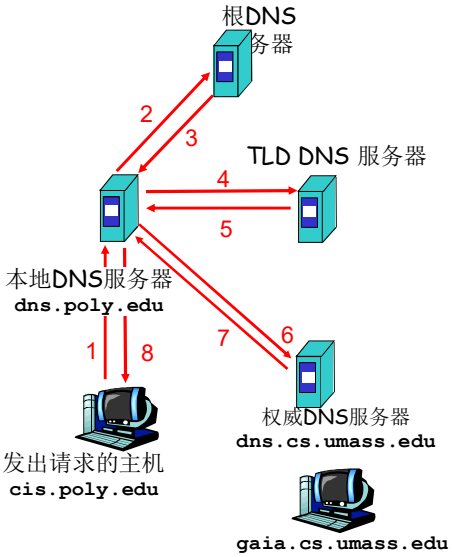
递归查询：

- 名字解析负担都放在当前联络的名字服务器上
- 问题：根服务器的负担太重
- 解决： 迭代查询 (iterated queries)



迭代查询

- 主机cis.poly.edu 想知道主机 gaia.cs.umass.edu 的IP地址



- 根（及各级域名）服务器返回的不是查询结果，而是下一个NS的地址
- 最后由权威名字服务器给出解析结果

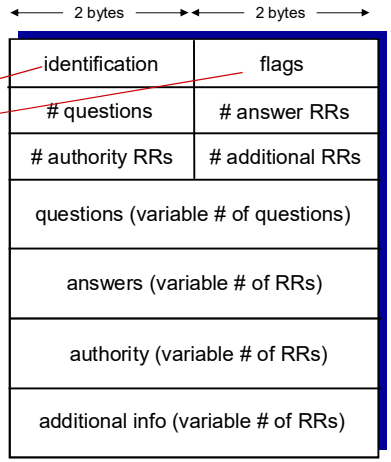
- 当前联络的服务器给出可以联系的服务器的名字
- “我不知道这个名字，但可以向这个服务器请求”

DNS协议、报文

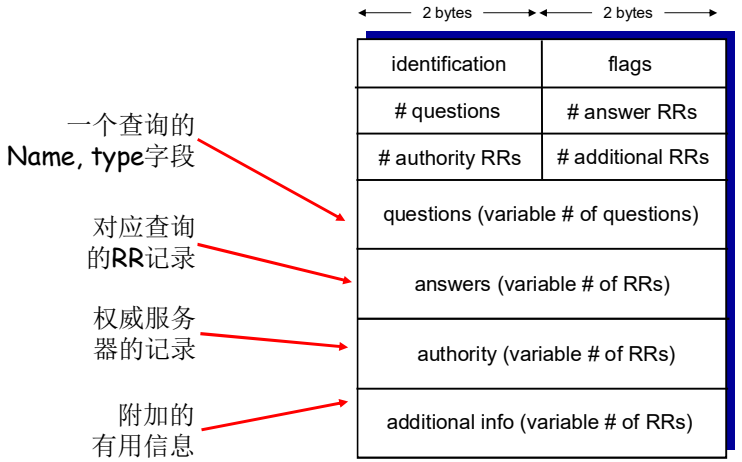
DNS协议：查询和响应报文的报文格式相同

报文首部

- 标识符（ID）：16位
- flags:
  - 查询/应答
  - 希望递归
  - 递归可用
  - 应答为权威



DNS协议、报文



提高性能：缓存

- 一旦名字服务器学到了一个映射，就将该映射缓存起来
- 根服务器通常都在本地服务器中缓存着
  - 使得根服务器不用经常被访问
- 目的：提高效率
- 可能存在的问题：如果情况变化，缓存结果和权威资源记录不一致
- 解决方案：TTL（默认2天）

问题3：维护问题：新增一个域

- 在上级域的名字服务器中增加两条记录，指向这个新增的子域的域名 和 域名服务器的地址
- 在新增子域 的名字服务器上运行名字服务器，负责本域的名字解析： 名字->IP地址

例子：在com域中建立一个“Network Utopia”

- 到注册登记机构注册域名networkutopia.com
  - 需要向该机构提供权威DNS服务器（基本的、和辅助的）的名字和IP地址
  - 登记机构在com TLD服务器中插入两条RR记录:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- 在networkutopia.com的权威服务器中确保有
  - 用于Web服务器的www.networkuptopia.com的类型为A的记录
  - 用于邮件服务器mail.networkutopia.com的类型为MX的记录

攻击DNS

总的说来，DNS比较健壮

DDoS 攻击

- 对根服务器进行流量轰炸攻击：发送大量ping
  - 没有成功
  - 原因 1 ：根目录服务器配置了流量过滤器，防火墙
  - 原因 2 ：Local DNS 服务器缓存了TLD服务器的IP地址，因此无需查询根服务器
- 向TLD服务器流量轰炸攻击：发送大量查询
  - 可能更危险
  - 效果一般，大部分DNS缓存了TLD

重定向攻击

- 中间人攻击
  - 截获查询，伪造回答，从而攻击某个（DNS回答指定的IP）站点
- DNS中毒
  - 发送伪造的应答给DNS服务器，希望它能够缓存这个虚假的结果
- 技术上较困难：分布式截获和伪造

利用DNS基础设施进行DDoS

- 伪造某个IP进行查询， 攻击这个目标IP
- 查询放大，响应报文比查询报文大
- 效果有限

第2章：提纲

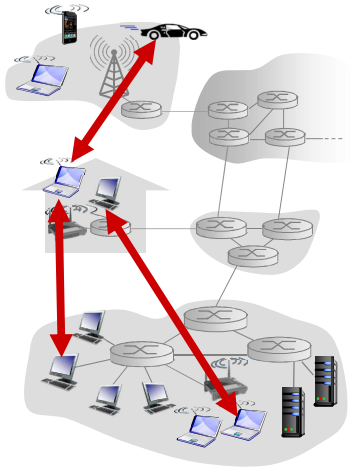
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

纯P2P架构

- 没有（或极少）一直运行的服务器
- 任意端系统都可以直接通信
- 利用peer的服务能力
- Peer节点间歇上网，每次IP地址都有可能变化

例子：

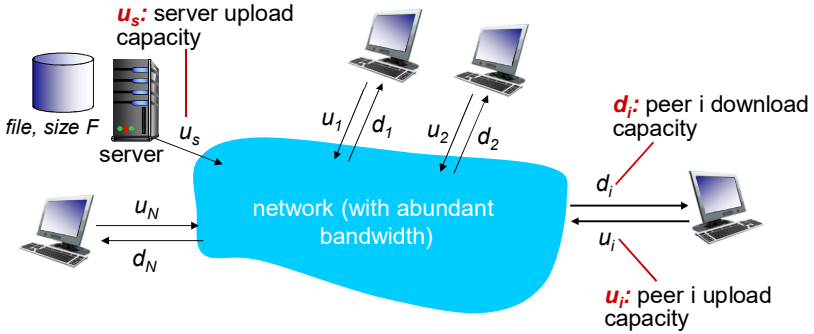
- 文件分发（BitTorrent）
- 流媒体(KanKan)
- VoIP（Skype）



文件分发：C/S vs P2P

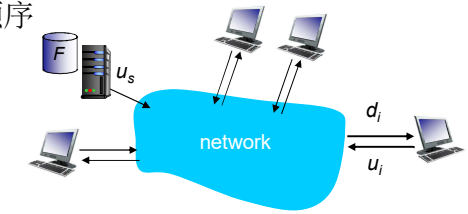
问题： 从一台服务器分发文件（大小F）到N个peer需要多少时间？

- Peer节点上下载能力是有限的资源



文件分发时间：C/S模式

- *服务器传输：* 都是由服务器发送给peer，服务器必须顺序传输（上载）N个文件拷贝：
  - 发送一个copy：F/u_s
  - 发送N个copy：NF/u_s
- *客户端：* 每个客户端必须下载一个文件拷贝
  - d_{min} = 客户端最小的下载速率
  - 下载带宽最小的客户端下载的时间：F/d_{min}

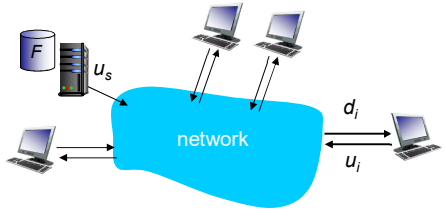


采用C-S方法将一个F大小的文件分发给N个客户端耗时 $D_{c\text{-}s} \geq max\{NF/u_s, F/d_{min}\}$

随着N线性增长

文件分发时间：P2P模式

- *服务器传输：* 最少需要上载一份拷贝
  - 发送一个拷贝的时间：F/u_s
- *客户端：* 每个客户端必须下载一个拷贝
  - 最小下载带宽客户单耗时：：F/d_{min}
- *客户端：* 所有客户端总体下载量NF
  - 最大上载带宽是：$u_s + \Sigma u_i$
  - 除了服务器可以上载，其他所有的peer节点都可以上载



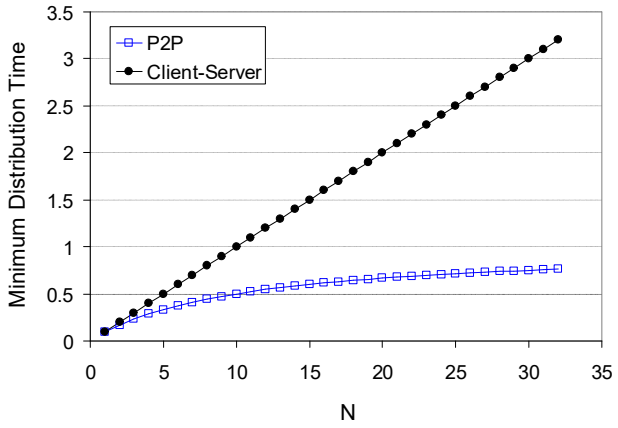
采用P2P方法将一个F大小的文件分发给N个客户端耗时 $D_{P2P} \geq max\{F/u_s, F/d_{min}, NF/(u_s + \Sigma u_i)\}$

分子随着N线性变化，每个节点需要下载，整体下载量随着N增大 …

… 分母也是如此，随着peer节点的增多每个peer也带了服务能力

Client-server vs. P2P：例子

客户端上载速率 = u， F/u = 1 小时， $u_s = 10u$， $d_{min} \geq u_s$

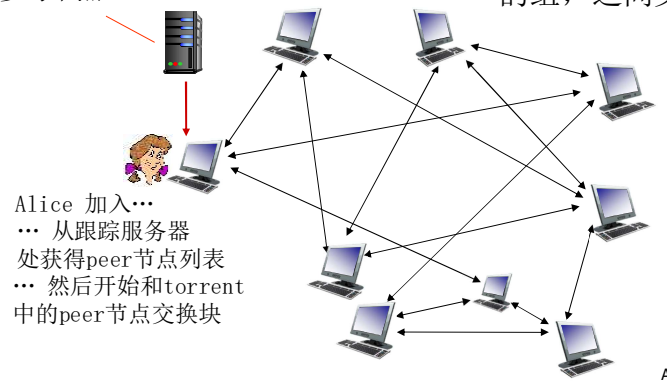


P2P文件分发： BitTorrent

- 文件被分为一个个块256KB
- 网络中的这些peers发送接收文件块，相互服务

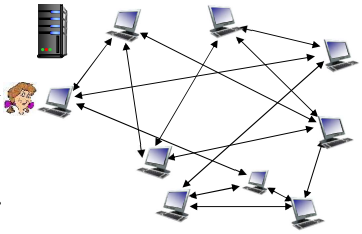
*tracker:*跟踪torrent中参与节点

Torrent（洪流）： 节点的组，之间交换文件块



P2P文件分发： BitTorrent

- Peer加入torrent：
  - 一开始没有块，但是将会通过其他节点处累积文件块
  - 向跟踪服务器注册，获得peer节点列表，和部分peer节点构成邻居关系（“连接”）



- 当peer下载时，该peer可以同时向其他节点提供上载服务
- Peer可能会变换用于交换块的peer节点
- *扰动churn:* peer节点可能会上线或者下线
- 一旦一个peer拥有整个文件，它会（自私的）离开或者保留（利他主义）在torrent中

BitTorrent：请求，发送文件块

请求块：

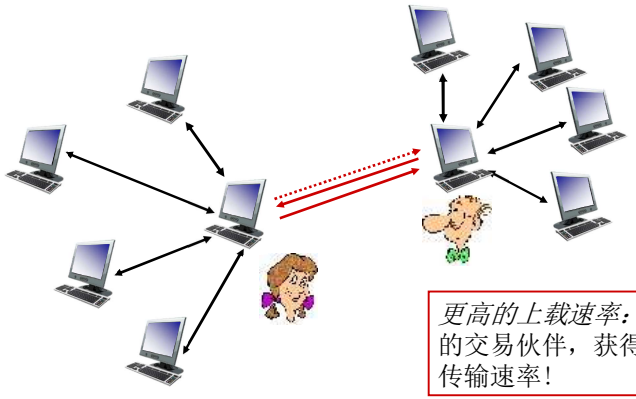
- 在任何给定时间，不同peer节点拥有一个文件块的子集
- 周期性的，Alice节点向邻居询问他们拥有哪些块的信息
- Alice向peer节点请求它希望的块，稀缺的块

发送块：一报还一报tit-for-tat

- Alice向4个peer发送块，这些块向它自己提供最大带宽的服务
  - 其他peer被Alice阻塞（将不会从Alice处获得服务）
  - 每10秒重新评估一次：前4位
- 每个30秒：随机选择其他peer节点，向这个节点发送块
  - “优化疏通” 这个节点
  - 新选择的节点可以加入这个top 4

BitTorrent: tit-for-tat

(1) Alice “优化疏通” Bob
(2) Alice 变成了Bob的前4位提供者；Bob答谢Alice
(3) Bob 变成了Alice的前4提供者



更高的上载速率： 发现更好的交易伙伴，获得更快的文件传输速率！

P2P文件共享

例子

- Alice在其笔记本电脑上运行P2P客户端程序
- 间歇性地连接到Internet，每次从其ISP得到新的IP地址
- 请求“双截棍.MP3”
- 应用程序显示其他有“双截棍.MP3” 拷贝的对等方
- Alice选择其中一个对等方，如Bob.
- 文件从Bob’s PC传送到Alice的笔记本上：HTTP
- 当Alice下载时，其他用户也可以从Alice处下载
- Alice的对等方既是一个Web客户端，也是一个瞬时Web服务器

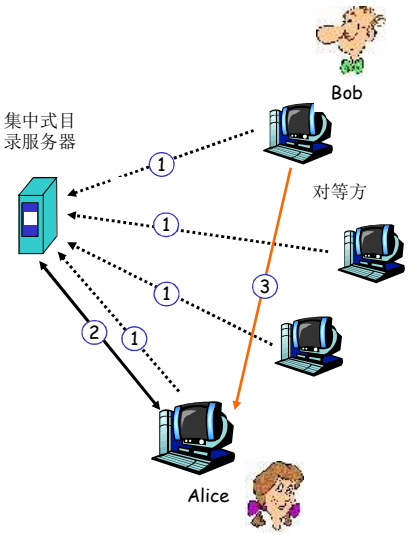
所有的对等方都是服务器
= 可扩展性好！

P2P文件共享

- 两大问题：
  - 如何定位所需资源
  - 如何处理对等方的加入与离开
- 可能的方案
  - 集中
  - 分散
  - 半分散

P2P：集中式目录

最初的“Napster”设计
1) 当对等方连接时，它告知中心服务器：
   - IP地址
   - 内容
2) Alice查询 “双截棍.MP3”
3) Alice从Bob处请求文件



P2P：集中式目录中存在的问题

- 单点故障
- 性能瓶颈
- 侵犯版权

文件传输是分散的，而定位内容则是高度集中的

查询洪泛：Gnutella

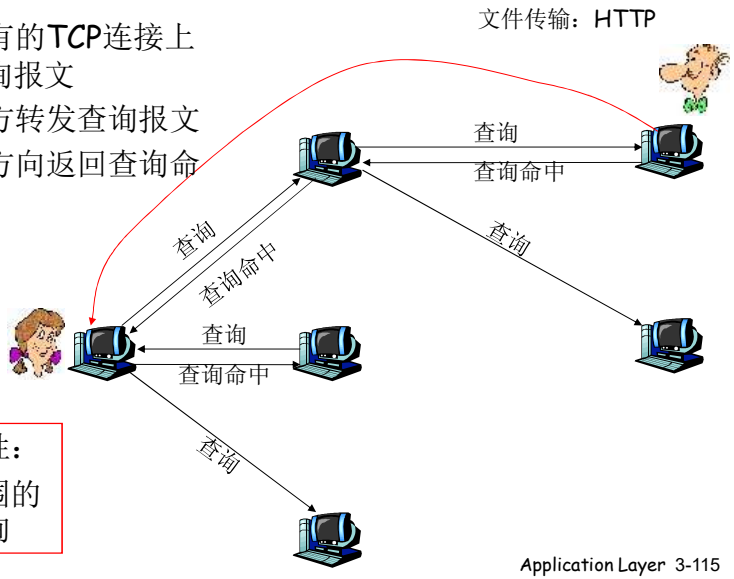
- 全分布式
  - 没有中心服务器
- 开放文件共享协议
- 许多Gnutella客户端实现了Gnutella协议
  - 类似HTTP有许多的浏览器

覆盖网络：图

- 如果X和Y之间有一个TCP连接，则二者之间存在一条边
- 所有活动的对等方和边就是覆盖网络
- 边并不是物理链路
- 给定一个对等方，通常所连接的节点少于10个

Gnutella：协议

- 在已有的TCP连接上发送查询报文
- 对等方转发查询报文
- 以反方向返回查询命中报文



可扩展性：
限制范围的洪泛查询

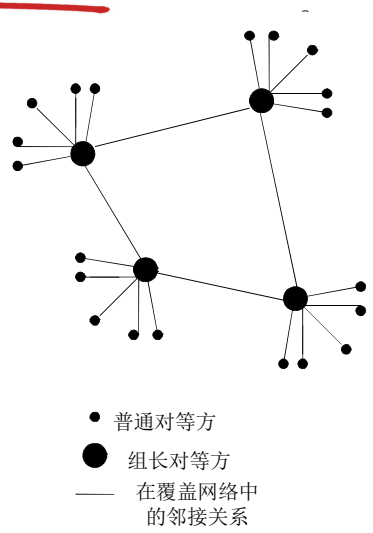
Gnutella：对等方加入

1. 对等方X必须首先发现某些已经在覆盖网络中的其他对等方：使用可用对等方列表
   自己维持一张对等方列表（经常开机的对等方的IP）
   联系维持列表的Gnutella站点
2. X接着试图与该列表上的对等方建立TCP连接，直到与某个对等方Y建立连接
3. X向Y发送一个Ping报文，Y转发该Ping报文
4. 所有收到Ping报文的对等方以Pong报文响应
   IP地址、共享文件的数量及总字节数
5. X收到许多Pong报文，然后它能建立其他TCP连接

对等方离开？

利用不匀称性：KaZaA

- 每个对等方要么是一个组长，要么隶属于一个组长
  - 对等方与其组长之间有TCP连接
  - 组长对之间有TCP连接
- 组长跟踪其所有的孩子的内容
- 组长与其他组长联系
  - 转发查询到其他组长
  - 获得其他组长的数据拷贝



KaZaA：查询

- 每个文件有一个散列标识码和一个描述符
- 客户端向其组长发送关键字查询
- 组长用匹配进行响应：
  - 对每个匹配：元数据、散列标识码和IP地址
- 如果组长将查询转发给其他组长，其他组长也以匹配进行响应
- 客户端选择要下载的文件
  - 向拥有文件的对等方发送一个带散列标识码的HTTP请求

Kazaa小技巧

- 请求排队
  - 限制并行上载的数量
  - 确保每个被传输的文件从上载节点接收一定量的带宽
- 激励优先权
  - 鼓励用户上载文件
  - 加强系统的扩展性
- 并行下载
  - 从多个对等方下载同一个文件的不同部分
    - HTTP的字节范围首部
    - 更快地检索一个文件

Distributed Hash Table (DHT)

- 哈希表
- DHT方案
- 环形DHT 以及覆盖网络
- Peer波动

第2章：提纲

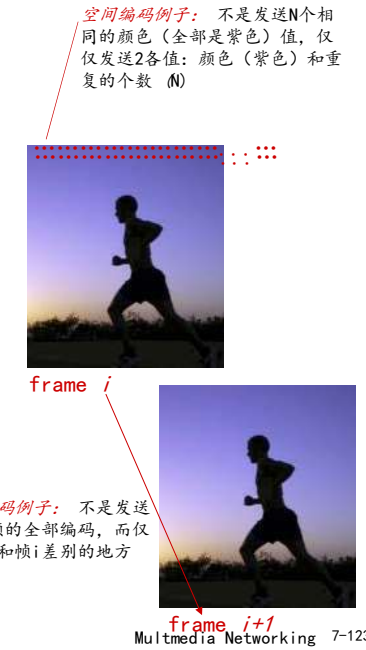
2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

视频流化服务和CDN：上下文

- 视频流量：占据着互联网大部分的带宽
  - Netflix，YouTube：占据37%，16% 的ISP下行流量
  - ~1B YouTube 用户，~75M Netflix用户
- 挑战：规模性–如何服务者 ~1B 用户？
  - 单个超级服务器无法提供服务（为什么）
- 挑战：异构性
  - 不同用户拥有不同的能力（例如：有线接入和移动用户；带宽丰富和受限用户）
- *解决方案：*分布式的，应用层面的基础设施

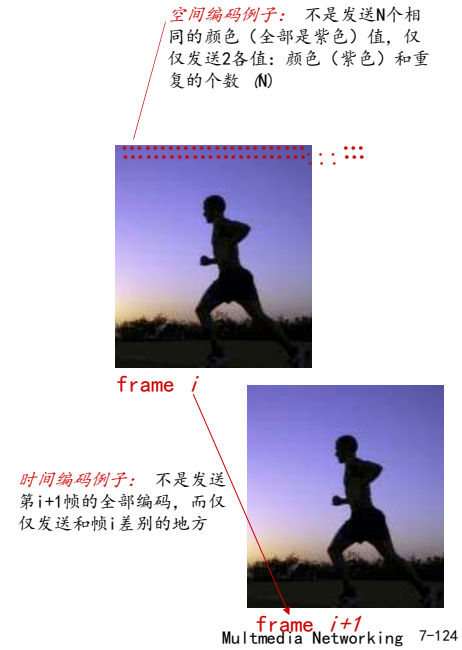
多媒体：视频

- 视频：固定速度显示的图像序列
  - e.g. 24 images/sec
- 网络视频特点：
  - 高码率：>10x于音频，高的网络带宽需求
  - 可以被压缩
  - 90%以上的网络流量是视频
- 数字化图像：像素的阵列
  - 每个像素被若干bits表示
- 编码：使用图像内和图像间的冗余来降低编码的比特数
  - 空间冗余（图像内）
  - 时间冗余（相邻的图像间）



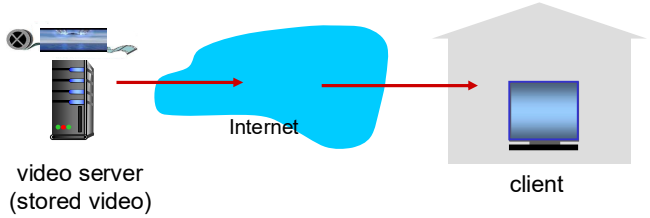
多媒体：视频

- CBR：(constant bit rate)：以固定速率编码
- VBR：(variable bit rate)：视频编码速率随时间的变化而变化
- 例子：
  - MPEG 1 (CD-ROM) 1.5 Mbps
  - MPEG2 (DVD) 3-6 Mbps
  - MPEG4 (often used in Internet, < 1 Mbps)



存储视频的流化服务：

简单场景：



多媒体流化服务：DASH

- DASH: Dynamic, Adaptive Streaming over HTTP
- *服务器：*
  - 将视频文件分割成多个块
  - 每个块独立存储，编码于不同码率（8-10种）
  - *告示文件（manifest file）：*提供不同块的URL
- *客户端：*
  - 先获取告示文件
  - 周期性地测量服务器到客户端的带宽
  - 查询告示文件，在一个时刻请求一个块，HTTP头部指定字节范围
    - 如果带宽足够，选择最大码率的视频块
    - 会话中的不同时刻，可以切换请求不同的编码块（取决于当时的可用带宽）

流式多媒体技术3：DASH

- “智能”客户端：客户端自适应决定
  - 什么时候去请求块（不至于缓存挨饿，或者溢出）
  - 请求什么编码速率的视频块（当带宽够用时，请求高质量的视频块）
  - 哪里去请求块（可以向离自己近的服务器发送URL，或者向高可用带宽的服务器请求）

Content Distribution Networks

- *挑战：*服务器如何通过网络向上百万用户同时流化视频内容（上百万视频内容)?
- *选择1：*单个的、大的超级服务中心“mega-server”
  - 服务器到客户端路径上跳数较多，瓶颈链路的带宽小导致停顿
  - “二八规律”决定了网络同时充斥着同一个视频的多个拷贝，效率低（付费高、带宽浪费、效果差）
  - 单点故障点，性能瓶颈
  - 周边网络的拥塞

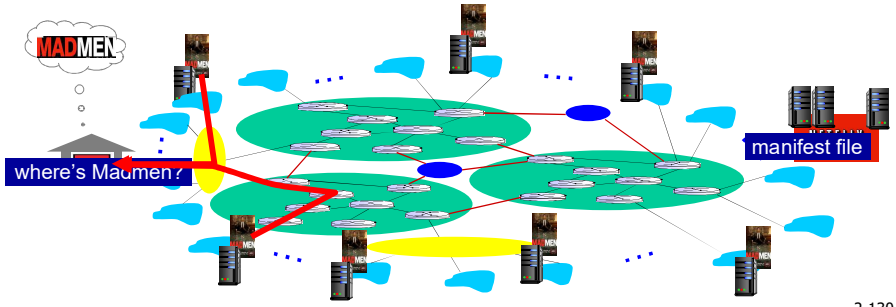
评述：相当简单，但是这个方法不可扩展

Content distribution networks

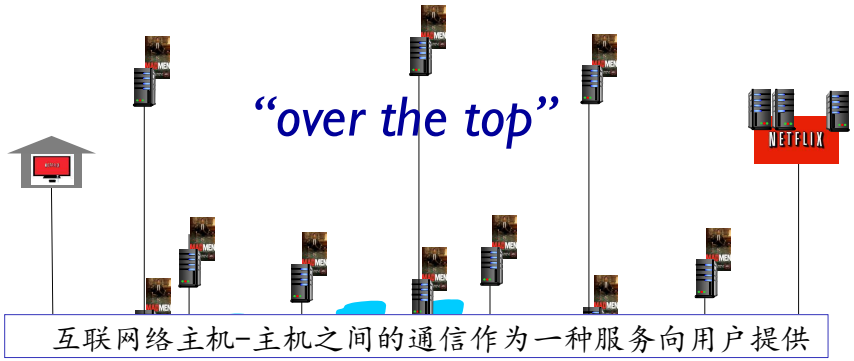
- *选项2:* 通过CDN，全网部署缓存节点，存储服务内容，就近为用户提供服务，提高用户体验
  - *enter deep:* 将CDN服务器深入到许多接入网
    - 更接近用户，数量多，离用户近，管理困难
    - Akamai，1700个位置
  - *bring home:* 部署在少数(10个左右)关键位置，如将服务器簇安装于POP附近（离若干1st ISP POP较近）
    - 采用租用线路将服务器簇连接起来
    - Limelight

Content Distribution Networks (CDNs)

- CDN：在CDN节点中存储内容的多个拷贝
  - e.g. Netflix stores copies of MadMen
- 用户从CDN中请求内容
  - 重定向到最近的拷贝，请求内容
  - 如果网络路径拥塞，可能选择不同的拷贝



Content Distribution Networks (CDNs)



OTT 挑战: 在拥塞的互联网上复制内容

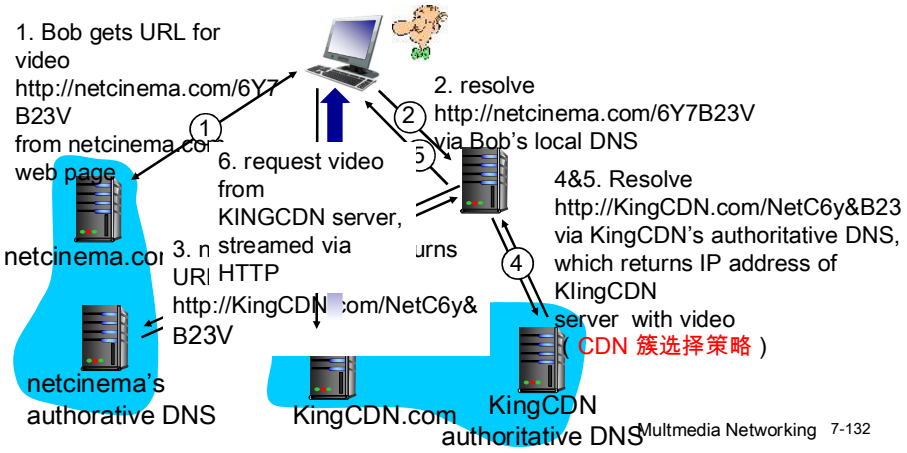
- 从哪个CDN节点中获取内容？
- 用户在网络拥塞时的行为？
- 在哪些CDN节点中存储什么内容？

more .. in chapter 7

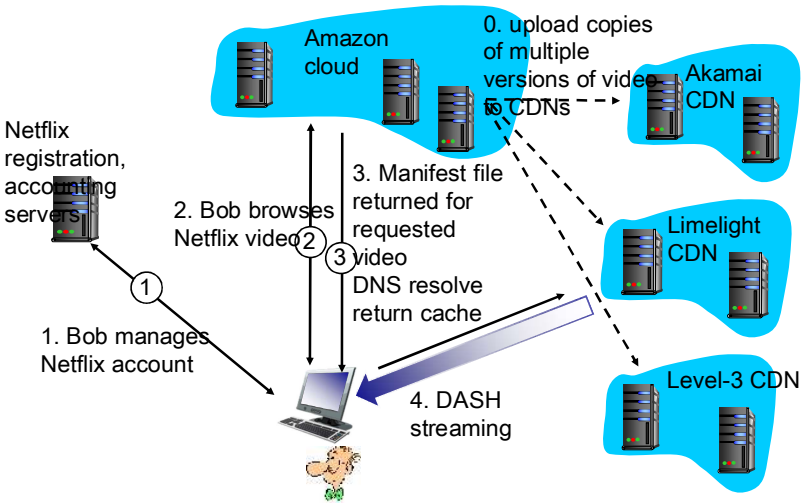
CDN："简单"内容访问场景

Bob(客户端)请求视频http://netcinema.com/6Y7B23V

- 视频存储在CDN，位于http://KingCDN.com/NetC6y&B23V



案例学习：Netflix



第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

Socket编程

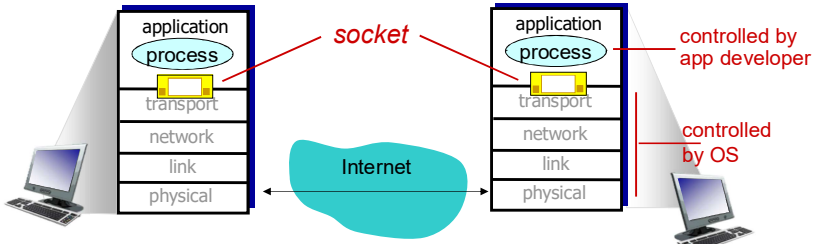
应用进程使用传输层提供的服务才能够交换报文，实现应用协议，实现应用

TCP/IP：应用进程使用Socket API访问传输服务

地点：界面上的SAP(Socket） 方式：Socket API

目标：学习如何构建能借助sockets进行通信的C/S应用程序

socket：分布式应用进程之间的门，传输层协议提供的端到端服务接口



Socket编程

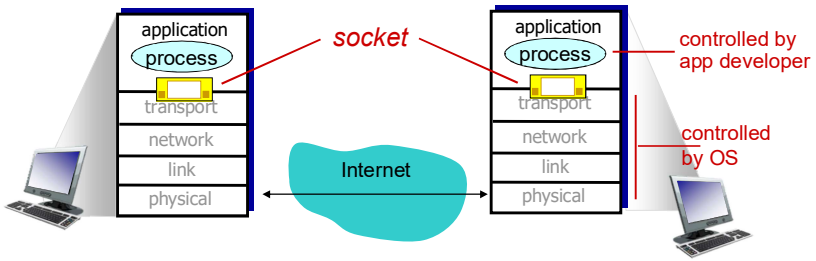
2种传输层服务的socket类型：

- *TCP:* 可靠的、字节流的服务
- *UDP:* 不可靠（数据UDP数据报）服务

TCP套接字编程

套接字：应用进程与端到端传输协议（TCP或UDP）之间的门户

TCP服务：从一个进程向另一个进程可靠地传输字节流



TCP套接字编程

服务器首先运行，等待连接建立

1：服务器进程必须先处于运行状态

- 创建欢迎socket
- 和本地端口捆绑
- 在欢迎socket上阻塞式等待接收用户的连接

客户端主动和服务器建立连接：

2：创建客户端本地套接字（隐式捆绑到本地port）

- 指定服务器进程的IP地址和端口号，与服务器进程连接

4：连接API调用有效时，客户端P与服务器建立了TCP连接

3：当与客户端连接请求到来时

- 服务器接受来自用户端的请求，解除阻塞式等待，返回一个新的socket（与欢迎socket不一样），与客户端通信
  - 允许服务器与多个客户端通信
  - 使用源IP和源端口来区分不同的客户端

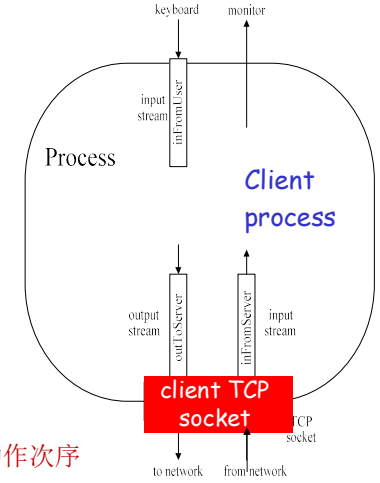
从应用程序的角度

TCP在客户端和服务器进程之间提供了可靠的、字节流（管道）服务

TCPsocket编程

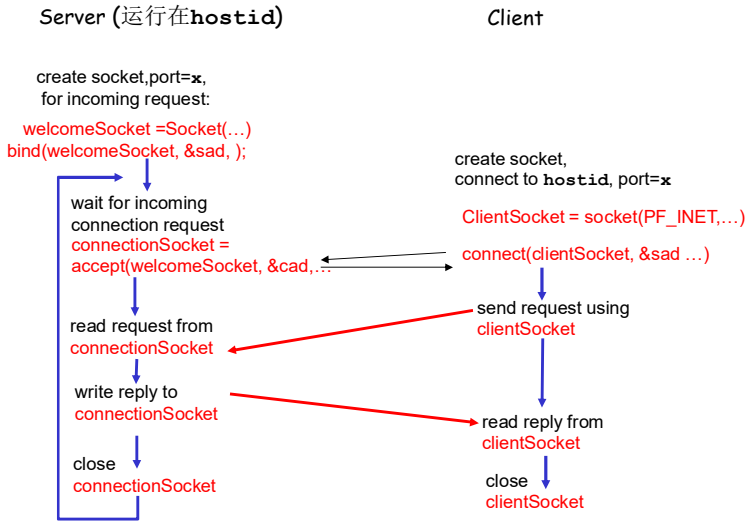
C/S模式的应用样例:

1) 客户端从标准输入装置读取一行字符，发送给服务器
2) 服务器从socket读取字符
3) 服务器将字符转换成大写，然后返回给客户端
4) 客户端从socket中读取一行字符，然后打印出来



实际上，这里描述了C-S之间交互的动作次序

C/S socket 交互: TCP



数据结构 sockaddr_in

IP地址和port捆绑关系的数据结构（标示进程的端节点）

```
struct sockaddr_in {
  short sin_family;    //AF_INET
  u_short sin_port;   // port
   struct in_addr sin_addr ;
          //  IP address, unsigned long
 char sin_zero[8]; // align
 };
```


数据结构 hostent

域名和IP地址的数据结构

```
struct hostent
    {      char *h_name;
           char **h_aliases;
           int h_addrtype;
           int h_length;        /*地址长度*/
           char **h_addr_list;
           #define h_addr h_addr_list[0];
     }
```

作为调用域名解析函数时的参数
返回后，将IP地址拷贝到 sockaddr_in的IP地址部分

例子: C客户端(TCP)

```
/* client.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address of server */
int clientSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry  */

char Sentence[128];
char modifiedSentence[128];

host = argv[1]; port = atoi(argv[2]);

clientSocket = socket(PF_INET, SOCK_STREAM, 0);
          memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
          sad.sin_family = AF_INET; /* set family to Internet */
          sad.sin_port = htons((u_short)port);
          ptrh = gethostbyname(host);
                   /* Convert host name to IP address */
          memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
                   //将IP地址拷贝到sad.sin_addr
       connect(clientSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create client socket, connect to server

例子: C客户端(TCP) （续）

```
gets(Sentence);

n=write(clientSocket, Sentence, strlen(Sentence)+1);

n=read(clientSocket, modifiedSentence, sizeof(modifiedSentence

printf("FROM SERVER: %s\n",modifiedSentence);

close(clientSocket);
}
```

Get input stream from user

Send line to server

Read line from server

Close connection

例子: C服务器（TCP）

```
/* server.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address  of server*/
struct sockaddr_in cad; /*client  */
int welcomeSocket, connectionSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char clientSentence[128];
char capitalizedSentence[128];

port = atoi(argv[1]);

welcomeSocket = socket(PF_INET, SOCK_STREAM, 0);
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
        sad.sin_port = htons((u_short)port);/* set the port number */
bind(welcomeSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create welcoming socket at port
&
Bind a local address

例子: C服务器（TCP）（续）

```
/* Specify the maximum number of clients that can be queued */
listen(welcomeSocket, 10)

while(1) {

    connectionSocket=accept(welcomeSocket, (struct sockaddr *)&cad, &alen);

    n=read(connectionSocket, clientSentence, sizeof(clientSentence));

    /* capitalize Sentence and store the result in capitalizedSentence*/


    n=write(connectionSocket, capitalizedSentence, strlen(capitalizedSentence)+1);

    close(connectionSocket);
    }
}
```

Wait, on welcoming socket for contact by a client

Write out the result to socket

End of while loop, loop back and wait for another client connection

第2章：提纲

2.1 应用层协议原理
2.2 Web and HTTP
2.3 FTP*
2.4 EMail
2.5 DNS
2.6 P2P 应用
2.7 CDN
2.8 TCP 套接字编程
2.9 UDP 套接字编程

UDP Socket编程

UDP: 在客户端和服务器之间没有连接

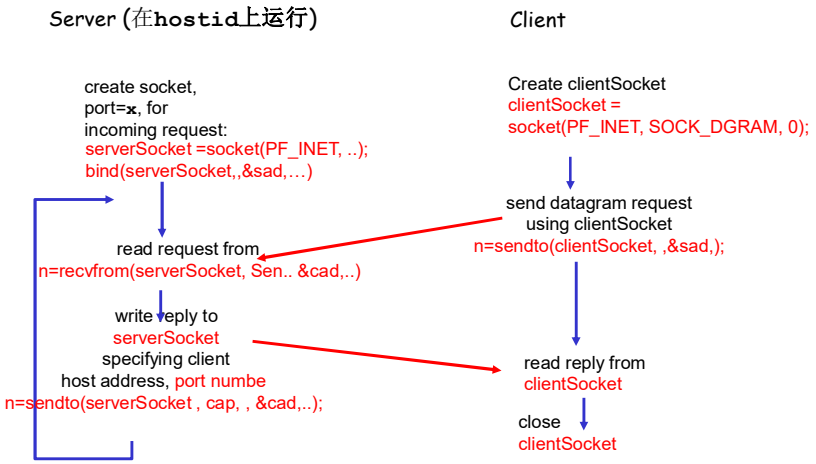
- 没有握手
- 发送端在每一个报文中明确地指定目标的IP地址和端口号
- 服务器必须从收到的分组中提取出发送端的IP地址和端口号

UDP: 传送的数据可能乱序，也可能丢失

进程视角看UDP服务

UDP 为客户端和服务器提供不可靠的字节组的传送服务

Client/server socket 交互: UDP



样例: C客户端 (UDP)

```
/* client.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address */
int clientSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char Sentence[128];
char modifiedSentence[128];

host = argv[1]; port = atoi(argv[2]);

clientSocket = socket(PF_INET, SOCK_DGRAM, 0);

/* determine the server's address */
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_port = htons((u_short)port);
        ptrh = gethostbyname(host);
            /* Convert host name to IP address */
        memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
```

创建客户端 socket,
没有连接到服务器

样例: C客户端 (UDP)（续）

```
Get input stream from user →  gets(Sentence);

Send line to server →  addr_len =sizeof(struct sockaddr);
                       n=sendto(clientSocket, Sentence, strlen(Sentence)+1,
                                (struct sockaddr *) &sad, addr_len);

Read line from server →  n=recvfrom(clientSocket, modifiedSentence, sizeof(modifiedSentence),
                                    (struct sockaddr *) &sad, &addr_len);

                         printf("FROM SERVER: %s\n",modifiedSentence);

Close connection →  close(clientSocket);
                    }
```


样例: C服务器(UDP)

```
/* server.c  */
void main(int argc, char *argv[])
{
struct sockaddr_in sad; /* structure to hold an IP address */
struct sockaddr_in cad;
int serverSocket; /* socket descriptor */
struct hostent *ptrh; /* pointer to a host table entry */

char clientSentence[128];
char capitalizedSentence[128];

port = atoi(argv[1]);

serverSocket = socket(PF_INET, SOCK_DGRAM, 0);
        memset((char *)&sad,0,sizeof(sad)); /* clear sockaddr structure */
        sad.sin_family = AF_INET; /* set family to Internet */
        sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */
        sad.sin_port = htons((u_short)port);/* set the port number */
bind(serverSocket, (struct sockaddr *)&sad, sizeof(sad));
```

Create welcoming socket at port
&
Bind a local address

样例: C服务器(UDP)（续）

```
while(1) {

   n=recvfrom(serverSocket, clientSentence, sizeof(clientSentence), 0
              (struct sockaddr *) &cad, &addr_len );

   /* capitalize Sentence and store the result in capitalizedSentence*/

   n=sendto(serverSocket , capitalizedSentence, strlen(capitalizedSentence)+1,
            (struct sockaddr *) &cad, &addr_len);

   }
}
```

Receive messages from clients

Write out the result to socket

End of while loop, loop back and wait for another client connection

第2章：小结

- 应用程序体系结构
  - 客户-服务器
  - P2P
  - 混合
- 应用程序需要的服务品质描述:
  - 可靠性、带宽、延时、安全
- Internet传输层服务模式
  - 可靠的、面向连接的服务：TCP
  - 不可靠的数据报：UDP
- 流行的应用层协议:
  - HTTP
  - FTP
  - SMTP, POP, IMAP
  - DNS
- Socket编程

第2章：小结

更重要的：学习协议的知识

- 应用层协议报文类型：请求/响应报文：
  - 客户端请求信息或服务
  - 服务器以数据、状态码进行响应
- 报文格式：
  - 首部：关于数据信息的字段
  - 数据：被交换的信息
- 控制报文 vs. 数据报文
  - 带内、带外
- 集中式 vs. 分散式
- 无状态 vs. 维护状态
- 可靠的 vs. 不可靠的报文传输
- 在网络边缘处理复杂性

一个协议定义了在两个或多个通信实体之间交换报文的格式和次序、以及就一条报文传输和接收或其他事件采取的动作